Володимирський педагогічний фаховий коледж імені Агатангела Кримського
циклова комісія викладачів професійної цифрової та математичної освіти

Інструкція до практичної роботи №60

ТЕМА: Створення анімованих графічних інтерфейсів з інтерактивним керуванням об'єктами.

Освітньо-професійна програма: Професійна освіта (Цифрові технології)
Тривалість заняття: 80 хв

Мета заняття:

- закріпити навички побудови анімації на CTkCanvas;
- навчитися керувати анімованими об'єктами за допомогою клавіатури та миші;
- ознайомитися з обробкою подій клавіатури через bind() на рівні вікна;
- розвинути навички поєднання стану програми (змінних) з візуальним відображенням у реальному часі.

Теоретичні відомості

1. Від автоматичної анімації до інтерактивного керування

У практичній роботі №59 об'єкт рухався автоматично, за наперед заданим правилом, без участі користувача. Інтерактивна графічна програма (і тим паче гра) вимагає іншого підходу: положення об'єкта повинно змінюватися у відповідь на дії користувача — натискання клавіш, рух миші, клацання по об'єктах. Технічно це означає, що ми поєднуємо вже відомий механізм обробки подій (bind(), тема 5) із вже відомим механізмом переміщення об'єктів на полотні (move(), тема 6).

2. Обробка подій клавіатури на рівні вікна

Щоб реагувати на натискання клавіш, обробник події прив'язують не до окремого віджета, а до головного вікна цілком — адже клавіатурні події мають сенс для програми загалом, а не для конкретного елемента інтерфейсу. Для цього вікно попередньо повинно мати фокус введення, який зазвичай встановлюють методом focus_set():

```
def on_key_press(event):
    print('Натиснуто клавішу:', event.keysym)

app.bind('<KeyPress>', on_key_press)
app.focus_set()
```

Атрибут event.keysym містить символьну назву натиснутої клавіші ('Left', 'Right', 'Up', 'Down', 'space', або звичайну літеру чи цифру) — саме за значенням цього атрибута програма визначає, яку саме дію потрібно виконати.

3. Керування об'єктом стрілками клавіатури

Об'єднавши обробку клавіатурних подій з методом move(), отримуємо базове керування об'єктом на полотні:

```
player = canvas.create_oval(180, 130, 220, 170, fill='#3a7ebf')

def on_key_press(event):
    if event.keysym == 'Left':
        canvas.move(player, -10, 0)
    elif event.keysym == 'Right':
        canvas.move(player, 10, 0)
    elif event.keysym == 'Up':
        canvas.move(player, 0, -10)
    elif event.keysym == 'Down':
        canvas.move(player, 0, 10)

app.bind('<KeyPress>', on_key_press)
app.focus_set()
```

4. Обмеження руху межами полотна

Без додаткових перевірок об'єкт, керований користувачем, може безконтрольно виїхати за межі видимої області полотна. Щоб цього уникнути, перед фактичним переміщенням перевіряють поточні координати об'єкта (метод coords() без додаткових аргументів повертає поточне положення) і виконують рух лише тоді, коли нова позиція залишається в межах дозволеної області:

```
def move_right():
    x1, y1, x2, y2 = canvas.coords(player)
    if x2 < 380:
        canvas.move(player, 10, 0)
```

Такий підхід — «перевір умову, і тільки тоді виконуй дію» — прямо повторює логіку умовного оператора з теми 2, застосовану тепер до графічного стану програми, а не до простих числових обчислень.

5. Реагування на клацання мишею по об'єкту

Окрім клавіатури, полотно може реагувати й на клацання мишею — причому подію можна прив'язати не до всього полотна, а до конкретного тегу чи об'єкта, щоб реагувати лише на клік саме по ньому, а не по довільному місцю полотна:

```
def on_player_click(event):
    canvas.itemconfig(player, fill='#e07a5f')

canvas.tag_bind(player, '<Button-1>', on_player_click)
```

Метод tag_bind() (на відміну від звичайного bind()) прив'язує обробник саме до конкретного об'єкта чи тегу на полотні — подія спрацює, тільки якщо клацання відбулося в межах цього конкретного об'єкта, а не будь-де на полотні.

6. Поєднання автоматичної анімації з керуванням користувача

Найцікавіші інтерактивні програми поєднують обидва підходи одночасно: одні об'єкти рухаються автоматично (через after(), як у темі 59), а інші керуються користувачем (через клавіатуру чи мишу) — і при цьому програма може перевіряти взаємне розташування цих об'єктів (наприклад, чи «зіткнулися» вони), що ми детальніше розглянемо в наступних практичних роботах.

Хід роботи

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Параметри руху залежать від N (mod — остача від ділення):

Завдання	Дані варіанта	Приклад N = 7
1	крок гравця S = 10 + 5·(N mod 3)	S = 15
4	швидкість другого об'єкта (2 + N mod 3, 2)	(3, 2)
6*	швидкість v = 4 + N mod 3	v = 5

б) Кожне завдання — окремий файл Прізвище_ПР60_зN.py (наприклад, Ivanenko_PR60_z3.py), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) Для кожного завдання додайте скріншот вікна із видимим заголовком. Імпорт — import customtkinter as ctk; програма завершується app.mainloop(). Тексти елементів і повідомлень — точно як в умові. Обробники клавіш прив'язуйте до вікна (app.bind), а не до полотна.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Створіть файл Прізвище_ПР60_з1.py. Створіть на полотні 400×300 коло-«гравця» радіусом 15 із центром (200, 150) і керуйте ним чотирма стрілками (bind до вікна, event.keysym) кроком S вашого варіанта.

   Приклад (N = 7): S = 15: після «←» центр (200 − 15, 150) = (185, 150).

2. Не дайте об'єкту виїхати за межі полотна (центр ± радіус у межах 0…400 і 0…300).

   Приклад: біля лівої межі «←» не зміщує коло.

3. За допомогою tag_bind() змінюйте колір кола при кліку мишею (по колу двох кольорів).

   Приклад: клік по колу — колір змінюється, клік повз — ні.

4. Додайте другий об'єкт, що автоматично рухається зі швидкістю (dx, dy) вашого варіанта через after() і відбивається від меж полотна.

   Приклад (N = 7): (dx, dy) = (3, 2).

5. Додайте CTkLabel «Координати: x=…, y=…» (центр гравця), що оновлюється після кожного руху.

   Приклад: на старті «Координати: x=200, y=150».

6. Завдання підвищеної складності*. Плавний рух: прапорці напрямку встановлюються в '<KeyPress>' і скидаються в '<KeyRelease>', а функція через after(20) зміщує гравця на v пікселів, доки клавішу утримують (v вашого варіанта).

   Приклад (N = 7): v = 5; після відпускання клавіші гравець зупиняється.

Критерії оцінювання

Складова	**Оцінювання**	**Макс. балів**
Завдання 1–5	кожне по 1,8 бала	9
Завдання 6*	2 бали	2
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, обробник прив'язано не до вікна, об'єкт виходить за межі полотна, рух перериваний чи дублюється кілька циклів after(), координати не оновлюються) — 50%; програма не працює або не здана — 0. Завдання 1–5 оцінюються по 1,8 бала, завдання 6* — 2 бали. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити крок руху чи межі полотна). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Чому обробник клавіатурних подій зазвичай прив'язують до всього вікна, а не до окремого віджета?
2. Що міститься в атрибуті event.keysym?
3. Як обмежити рух об'єкта так, щоб він не виїжджав за межі полотна?
4. Чим tag_bind() відрізняється від звичайного bind()?
5. Як реалізувати плавний, безперервний рух об'єкта, доки утримується клавіша, а не один крок на одне натискання?